

Tarefa 8: Coerência de Cache e Falso Compartilhamento

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Este relatório analisa quatro versões de um programa em C/OpenMP que estima π pelo método de Monte Carlo: sorteiam-se pontos (x, y) no quadrado $[0, 1] \times [0, 1]$ e verifica-se quantos caem dentro do quarto de círculo unitário ($x^2 + y^2 \leq 1$); a razão entre acertos e total de pontos converge para $\pi/4$.

O objetivo é comparar duas formas de acumular os acertos de cada thread – variável privada somada a um total global via `#pragma omp critical`, ou vetor compartilhado com uma posição por thread e soma serial ao final – combinadas com dois geradores de números aleatórios: `rand()` (estado global, não thread-safe por padrão) e um gerador com estado privado por thread. A análise explica os tempos observados com base em coerência de cache e falso compartilhamento (*false sharing*).

2 Metodologia

O programa implementa quatro variantes, todas com o laço de amostragem distribuído via `#pragma omp for`:

1. **V1 – `rand()` + `critical`**: contador local por thread, somado ao total global em um bloco `critical` ao final da região paralela.
2. **V2 – `rand()` + `vetor`**: cada thread grava seu contador local em `acertos_por_thread[id]`; a soma é feita em laço serial após a região paralela.
3. **V3 – `Xorshift32` + `critical`**: igual a V1, trocando `rand()` por um gerador `Xorshift32` com semente privada por thread.
4. **V4 – `Xorshift32` + `vetor`**: igual a V2, também com `Xorshift32`.

Como o ambiente é Windows e `rand_r()` é uma função POSIX indisponível no MinGW/MSVC, ela foi substituída por um `Xorshift32` com estado (semente) puramente privado por thread – preservando a característica central que se quer testar: um gerador sem estado global compartilhado.

O programa foi executado com 8 threads, para n_{pontos} de 10^5 a 10^9 (passos $\times 10$), medindo o tempo de cada versão com `omp_get_wtime()`. O código completo está no Apêndice A.

3 Resultados

```

=====
ESTIMATIVA DE PI - METODO DE MONTE CARLO
=====
PI_REAL = 3.141592653589793
Threads = 8
Intervalo de n_pontos = 100000 ate 10000000000

N_PONTOS    VERSAO  PI_MEDIDO  ERRO_ABS  ERRO_REL(%)  TEMPO(s)
-----
100000      1       3.11740000  0.02419265  0.770076     0.0020
100000      2       3.13556000  0.00603265  0.192025     0.0010
100000      3       3.13220000  0.00939265  0.298977     0.0000
100000      4       3.13220000  0.00939265  0.298977     0.0000
-----
1000000     1       3.13700000  0.00459265  0.146189     0.0060
1000000     2       3.14625600  0.00466335  0.148439     0.0050
1000000     3       3.14120800  0.00038465  0.012244     0.0010
1000000     4       3.14120800  0.00038465  0.012244     0.0000
-----
10000000    1       3.14031280  0.00127985  0.040739     0.0990
10000000    2       3.13953440  0.00205825  0.065516     0.0710
10000000    3       3.14231040  0.00071775  0.022847     0.0100
10000000    4       3.14231040  0.00071775  0.022847     0.0090
-----
100000000   1       3.14113016  0.00046249  0.014722     0.4030
100000000   2       3.14198412  0.00039147  0.012461     0.4460
100000000   3       3.14182792  0.00023527  0.007489     0.0890
100000000   4       3.14182792  0.00023527  0.007489     0.0830
-----
1000000000  1       3.14143575  0.00015691  0.004994     4.0620
1000000000  2       3.14160874  0.00001609  0.000512     4.1530
1000000000  3       3.14157712  0.00001554  0.000495     0.7930
1000000000  4       3.14157712  0.00001554  0.000495     0.7610
-----

```

Código 1: Saída da execução com 8 threads

Tabela 1: Tempo de execução (s) por versão e por tamanho de amostra

N_PONTOS	V1: rand+critical	V2: rand+vetor	V3: xorshift+critical	V4: xorshift+vetor
10^5	0,0020	0,0010	0,0000	0,0000
10^6	0,0060	0,0050	0,0010	0,0000
10^7	0,0990	0,0710	0,0100	0,0090
10^8	0,4030	0,4460	0,0890	0,0830
10^9	4,0620	4,1530	0,7930	0,7610

O erro relativo cai com o aumento de n_{pontos} em todas as versões, como esperado de um estimador cuja incerteza decresce com $1/\sqrt{n}$. As quatro versões convergem para o mesmo π ; a diferença relevante está nos tempos de execução.

4 Análise

rand() vs. Xorshift32. Esse é o fator que mais pesa nos tempos. Para $n_pontos = 10^9$, V1 e V2 (**rand()**) levam cerca de 4,1s, contra apenas 0,76–0,79s de V3 e V4 (Xorshift32) – uma diferença de $\sim 5,3\times$. A razão é que **rand()** mantém um único estado global, compartilhado por todas as threads, *sem nenhuma proteção explícita* contra acesso concorrente – as chamadas simultâneas a **rand()** em V1 e V2 constituem, a rigor, uma condição de corrida, já que múltiplas threads leem e escrevem a mesma variável de estado sem sincronização. Como o laço chama **rand()** duas vezes por iteração, esse estado é lido/escrito centenas de milhões de vezes, e cada escrita obriga o protocolo de coerência de cache a invalidar e retransmitir a mesma linha de cache entre os núcleos (um caso de compartilhamento verdadeiro).

Isso serializa boa parte do trabalho que deveria ser paralelo – não porque exista uma seção crítica formal protegendo o estado, mas porque o hardware precisa manter a coerência da linha de cache a cada acesso concorrente. Vale notar que essa condição de corrida em V1 e V2 é, tecnicamente, comportamento indefinido pela linguagem C; os resultados de π obtidos continuam estatisticamente razoáveis porque o método de Monte Carlo tolera ruído nos números gerados, mas isso não deve ser confundido com um padrão de acesso seguro. O Xorshift32, por outro lado, guarda sua semente como variável totalmente privada (registrador/pilha de cada thread), sem estado compartilhado, e por isso escala quase linearmente com o número de threads.

critical vs. vetor compartilhado. Comparando V1/V2 e V3/V4, as diferenças são pequenas (ex.: 0,793s vs. 0,761s em V3/V4 para 10^9). Isso ocorre porque, em ambas as versões, o contador é acumulado localmente (**acertos_local**, em registrador) durante todo o laço de n_pontos iterações; a sincronização – seção crítica ou escrita no vetor – acontece apenas *uma vez por thread*, ao final. Como esse custo é $O(1)$ por thread e não $O(n_pontos)$, ele é irrelevante frente ao custo de gerar e testar centenas de milhões de pontos, e por isso a escolha entre as duas estratégias pouco altera o tempo total.

Por que quase não há falso compartilhamento. O vetor **acertos_por_thread** é de `long long` (8 bytes); como uma linha de cache tem 64 bytes, até 8 contadores de threads vizinhas compartilham a mesma linha. Se cada thread escrevesse em **acertos_por_thread[id]** a cada iteração do laço, haveria falso compartilhamento severo (invalidações constantes da linha entre threads que não compartilham dado algum, apenas a linha de cache). Mas no código essa escrita ocorre só *uma vez*, após o laço paralelo – então o número de possíveis invalidações é da ordem do número de threads (8), não de n_pontos (até 10^9). É por isso que V2 e V4 não são mais lentas que V1 e V3: o padrão de acesso ao vetor é raro demais para que o falso compartilhamento apareça no tempo medido.

Resumo. Para 10^9 pontos: $V4 (0,761s) < V3 (0,793s) \ll V1 (4,062s) < V2 (4,153s)$. Os dois níveis de desempenho, bem separados, mostram que o gerador aleatório (estado global vs. privado) é o fator dominante; a estratégia de acumulação (**critical** vs. vetor) é secundária.

5 Conclusão

O gargalo de desempenho não está na acumulação dos resultados parciais, mas na geração dos números aleatórios. Um gerador com estado global (**rand()**), acessado concorrentemente sem qualquer proteção, sofre uma condição de corrida que gera tráfego intenso de coerência de cache entre os núcleos e serializa boa parte do programa. Um gerador com

estado privado por thread elimina esse problema e escala quase linearmente.

Já `critical` e vetor compartilhado se mostraram equivalentes, pois ambos custam apenas uma operação por thread, e não uma por iteração – inclusive o vetor, apesar de compartilhar linhas de cache entre threads, não sofre falso compartilhamento relevante por ser acessado tão raramente. A lição geral é que o estado verdadeiramente compartilhado (aqui, o gerador de números aleatórios) é o que determina a escalabilidade, muito mais do que pequenas seções críticas ou vetores usados com moderação.

A Código-Fonte

O código-fonte completo utilizado nesta atividade está disponível abaixo.

```
1  /* =====
2  * Estimativa estocastica de PI (metodo de Monte Carlo) com OpenMP
3  *
4  * Sao implementadas 4 versoes:
5  * 1) rand() + acumulacao via variavel privada + #pragma omp critical
6  * 2) rand() + acumulacao via vetor compartilhado + soma serial
7  * 3) xorshift32() + acumulacao via variavel privada + #pragma omp critical
8  * 4) xorshift32() + acumulacao via vetor compartilhado + soma serial
9  *
10 * Como o ambiente e Windows (sem rand_r() do POSIX), o gerador rand_r()
11 * foi substituido por um Xorshift32 thread-safe (estado privado por
12 * thread), que cumpre o mesmo papel: gerar numeros aleatorios sem
13 * depender de um estado global compartilhado.
14 *
15 * Compilar (MinGW / GCC no Windows):
16 * gcc -O2 -fopenmp pi_monte_carlo_omp.c -o pi_monte_carlo_omp.exe
17 *
18 * Executar:
19 * pi_monte_carlo_omp.exe [num_pontos] [num_threads]
20 * ===== */
21
22
23 #include <stdio.h>
24 #include <stdlib.h>
25 #include <math.h>
26 #include <time.h>
27 #include <limits.h>
28 #include <omp.h>
29
30 #define PI_REAL 3.14159265358979323846
31
32 #define MAX_THREADS 128
33
34 /* -----
35 * Gerador Xorshift32 (thread-safe)
36 * ----- */
37 static inline unsigned int xorshift32(unsigned int *state)
38 {
39     unsigned int x = *state;
40
41     x ^= x << 13;
42     x ^= x >> 17;
43     x ^= x << 5;
44
45     *state = x;
46     return x;
47 }
48
49 /* Geracao de double aleatorio */
50 static inline double random_double(unsigned int *state)
51 {
52     return (double)xorshift32(state) / (double)UINT_MAX;
53 }
54
```

```

55  /* Cria uma semente diferente por thread e NUNCA igual a zero */
56  static inline unsigned int semente_da_thread(void)
57  {
58      unsigned int base = (unsigned int)time(NULL) ^
59                          ((unsigned int)omp_get_thread_num() * 2654435761u);
60      return base | 1u;
61  }
62
63  /* -----
64  * VERSAO 1: rand() + contador privado + acumulaco global via critical
65  * ----- */
66  double pi_rand_critical(long long n_pontos, int n_threads)
67  {
68      long long total_acertos = 0;
69
70      #pragma omp parallel num_threads(n_threads)
71      {
72          long long acertos_local = 0;
73          long long i;
74
75          #pragma omp for
76          for (i = 0; i < n_pontos; i++)
77          {
78              double x = (double)rand() / (double)RAND_MAX;
79              double y = (double)rand() / (double)RAND_MAX;
80
81              if (x * x + y * y <= 1.0)
82              {
83                  acertos_local++;
84              }
85          }
86
87          /* Acumulaco do total */
88          #pragma omp critical
89          {
90              total_acertos += acertos_local;
91          }
92      }
93
94      return 4.0 * (double)total_acertos / (double)n_pontos;
95  }
96
97  /* -----
98  * VERSAO 2: rand() + vetor compartilhado + soma serial apos a regiao paralela
99  * ----- */
100 double pi_rand_vetor(long long n_pontos, int n_threads)
101 {
102     long long acertos_por_thread[MAX_THREADS] = {0};
103
104     #pragma omp parallel num_threads(n_threads)
105     {
106         int id = omp_get_thread_num();
107         long long acertos_local = 0;
108         long long i;
109
110         #pragma omp for
111         for (i = 0; i < n_pontos; i++)
112         {

```

```

113     double x = (double)rand() / (double)RAND_MAX;
114     double y = (double)rand() / (double)RAND_MAX;
115
116     if (x * x + y * y <= 1.0)
117     {
118         acertos_local++;
119     }
120 }
121
122 acertos_por_thread[id] = acertos_local;
123 }
124
125 long long total_acertos = 0;
126 for (int i = 0; i < n_threads; i++)
127 {
128     total_acertos += acertos_por_thread[i];
129 }
130
131 return 4.0 * (double)total_acertos / (double)n_pontos;
132 }
133
134 /* -----
135  * VERSAO 3: xorshift32() + contador privado + acumulador global via critical
136  * ----- */
137 double pi_xorshift_critical(long long n_pontos, int n_threads)
138 {
139     long long total_acertos = 0;
140
141     #pragma omp parallel num_threads(n_threads)
142     {
143         unsigned int seed = semente_da_thread();
144         long long acertos_local = 0;
145         long long i;
146
147         #pragma omp for
148         for (i = 0; i < n_pontos; i++)
149         {
150             double x = random_double(&seed);
151             double y = random_double(&seed);
152
153             if (x * x + y * y <= 1.0)
154             {
155                 acertos_local++;
156             }
157         }
158
159         #pragma omp critical
160         {
161             total_acertos += acertos_local;
162         }
163     }
164
165     return 4.0 * (double)total_acertos / (double)n_pontos;
166 }
167
168 /* -----
169  * VERSAO 4: xorshift32() + vetor compartilhado + soma serial
170  * ----- */

```

```

171 double pi_xorshift_vetor(long long n_pontos, int n_threads)
172 {
173     long long acertos_por_thread[MAX_THREADS] = {0};
174
175     #pragma omp parallel num_threads(n_threads)
176     {
177         int id = omp_get_thread_num();
178         unsigned int seed = semente_da_thread();
179         long long acertos_local = 0;
180         long long i;
181
182         #pragma omp for
183         for (i = 0; i < n_pontos; i++)
184         {
185             double x = random_double(&seed);
186             double y = random_double(&seed);
187
188             if (x * x + y * y <= 1.0)
189             {
190                 acertos_local++;
191             }
192         }
193
194         acertos_por_thread[id] = acertos_local;
195     }
196
197     long long total_acertos = 0;
198     for (int i = 0; i < n_threads; i++)
199     {
200         total_acertos += acertos_por_thread[i];
201     }
202
203     return 4.0 * (double)total_acertos / (double)n_pontos;
204 }
205
206 /* -----
207  * MAIN: executa e cronometra as 4 versoes
208  * ----- */
209 int main(int argc, char *argv[])
210 {
211     long long n_inicial = 100000;
212     long long n_final = 1000000000;
213     int n_threads = omp_get_max_threads();
214
215     if (argc >= 2) n_inicial = atoll(argv[1]);
216     if (argc >= 3) n_final = atoll(argv[2]);
217     if (argc >= 4) n_threads = atoi(argv[3]);
218
219     if (n_inicial <= 0 || n_final < n_inicial)
220     {
221         fprintf(stderr, "Intervalo de pontos invalido.\n");
222         fprintf(stderr, "Uso: %s [n_inicial] [n_final] [n_threads]\n", argv[0]);
223         return 1;
224     }
225
226     if (n_threads <= 0)
227     {
228         fprintf(stderr, "Numero de threads invalido.\n");

```



```

229     return 1;
230 }
231
232 if (n_threads > MAX_THREADS)
233 {
234     fprintf(stderr, "Numero de threads limitado a %d.\n", MAX_THREADS);
235     n_threads = MAX_THREADS;
236 }
237
238 srand((unsigned int)time(NULL));
239
240 printf("=====\n");
241 printf("      ESTIMATIVA DE PI - METODO DE MONTE CARLO\n");
242 printf("=====\n");
243 printf("PI_REAL = %.15f\n", PI_REAL);
244 printf("Threads = %d\n", n_threads);
245 printf("Intervalo de n_pontos = %lld ate %lld\n\n", n_inicial, n_final);
246
247 printf("%-12s %-8s %-12s %-12s %-12s %-12s\n",
248        "N_PONTOS", "VERSAO", "PI_MEDIDO", "ERRO_ABS", "ERRO_REL(%)", "TEMPO(s)");
249 printf("-----\n");
250
251 for (long long n_pontos = n_inicial;
252      n_pontos <= n_final;
253      )
254 {
255     double pi_medido;
256     double erro_abs;
257     double erro_rel;
258     double t0, t1;
259
260     /* =====
261      * VERSAO 1: rand() + critical
262      * ===== */
263     t0 = omp_get_wtime();
264     pi_medido = pi_rand_critical(n_pontos, n_threads);
265     t1 = omp_get_wtime();
266
267     erro_abs = fabs(pi_medido - PI_REAL);
268     erro_rel = (erro_abs / PI_REAL) * 100.0;
269
270     printf("%-12lld %-8d %-12.8f %-12.8f %-12.6f %-12.4f\n",
271           n_pontos, 1, pi_medido, erro_abs, erro_rel, t1 - t0);
272
273     /* =====
274      * VERSAO 2: rand() + vetor
275      * ===== */
276     t0 = omp_get_wtime();
277     pi_medido = pi_rand_vetor(n_pontos, n_threads);
278     t1 = omp_get_wtime();
279
280     erro_abs = fabs(pi_medido - PI_REAL);
281     erro_rel = (erro_abs / PI_REAL) * 100.0;
282
283     printf("%-12lld %-8d %-12.8f %-12.8f %-12.6f %-12.4f\n",
284           n_pontos, 2, pi_medido, erro_abs, erro_rel, t1 - t0);
285

```

```

286  /* =====
287  * VERSAO 3: xorshift32() + critical
288  * ===== */
289  t0 = omp_get_wtime();
290  pi_medido = pi_xorshift_critical(n_pontos, n_threads);
291  t1 = omp_get_wtime();
292
293  erro_abs = fabs(pi_medido - PI_REAL);
294  erro_rel = (erro_abs / PI_REAL) * 100.0;
295
296  printf("%-12lld %-8d %-12.8f %-12.8f %-12.6f %-12.4f\n",
297         n_pontos, 3, pi_medido, erro_abs, erro_rel, t1 - t0);
298
299  /* =====
300  * VERSAO 4: xorshift32() + vetor
301  * ===== */
302  t0 = omp_get_wtime();
303  pi_medido = pi_xorshift_vetor(n_pontos, n_threads);
304  t1 = omp_get_wtime();
305
306  erro_abs = fabs(pi_medido - PI_REAL);
307  erro_rel = (erro_abs / PI_REAL) * 100.0;
308
309  printf("%-12lld %-8d %-12.8f %-12.8f %-12.6f %-12.4f\n",
310         n_pontos, 4, pi_medido, erro_abs, erro_rel, t1 - t0);
311
312  printf("
313         -----\n");
314
315  /* Evita overflow ao multiplicar por 10. */
316  if (n_pontos > n_final / 10)
317      break;
318
319  n_pontos *= 10;
320  }
321
322  printf("\nLegenda:\n");
323  printf(" ERRO_ABS    = |PI_MEDIDO - PI_REAL|\n");
324  printf(" ERRO_REL    = (ERRO_ABS / PI_REAL) * 100\n");
325
326  return 0;
327 }

```

Código 2: Código-fonte completo (pi_monte_carlo_omp.c)